Aula 06 — Pré-processamento e *Pipelines*

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Leitura recomendada: ISLP labs cap. 5–6; AME §2.1.2

O que você vai aprender

Ao final desta aula você deve ser capaz de:

- explicar por que o pré-processamento faz parte do *modelo*, e não dos dados;
- definir a **padronização** (`StandardScaler`) e listar suas vantagens e desvantagens;
- identificar e evitar o **vazamento de dados** (*data leakage*), e saber *quais* vazamentos são graves;
- construir um ***pipeline*** que encadeia pré-processamento e estimador como um único objeto;
- combinar *pipeline* com validação cruzada e busca de hiperparâmetros sem vazamento;
- conhecer outros pré-processamentos comuns (imputação, codificação de categóricas) e o `ColumnTransformer`.

Esta é a aula que costura as anteriores em *boas práticas de implementação*. Vários métodos já exigiram padronização — Lasso e Ridge (Aula 02), KNN (Aula 04) — e a Aula 03 já avisou sobre vazamento. Aqui formalizamos *como* fazer isso certo. Comece pela pergunta:

onde, exatamente, eu devo calcular a média e o desvio para padronizar:
antes ou depois de separar treino e teste?

A resposta canônica é "depois", e você vai vê-la justificada na Seção 3. Mas a Figura 1 vai mostrar que, *nesse caso específico*, a diferença é praticamente nula — e que o vazamento que realmente destrói uma análise é outro. Vale ler até lá antes de tirar conclusões.

1 Motivação: o pré-processamento é parte do modelo

Na prática, raramente alimentamos o estimador com os dados brutos. Um fluxo típico encadeia várias transformações: padronizar, imputar faltantes, codificar variáveis categóricas, reduzir dimensão (PCA), e só então ajustar o modelo. A tentação é aplicar essas transformações "de uma vez" no conjunto inteiro — e é exatamente aí que mora o erro.

Ideia central

O insight central da aula: **toda transformação que "aprende" algo dos dados** (uma média, um desvio, as categorias presentes, os componentes principais) é *parte do procedimento de estimação*. Logo, deve ser *ajustada apenas no treino* e *aplicada* ao teste/validação — nunca o contrário.

Um teste mental que resolve quase todos os casos: pergunte-se *"essa etapa usa os dados para decidir alguma coisa?"*. Tomar o logaritmo de uma coluna, não — é a mesma função para qualquer conjunto. Subtrair a média, sim. Escolher as 20 variáveis mais correlacionadas com y, sim, e muito. Quanto mais a etapa "olha" para os dados, mais ela pertence ao modelo.

2 O `StandardScaler` (padronização)

Cada coluna $\boldsymbol{X}_i$ de uma base é uma amostra de uma variável aleatória X_i com $\mu_i = \mathbb{E}[X_i]$ e $\sigma_i^2 = \mathrm{Var}(X_i)$ (supostos finitos). A **padronização** transforma

$$\widetilde{X}_i = \frac{X_i - \mu_i}{\sigma_i}, \tag{1}$$

de modo que cada covariável passa a ter média 0 e variância 1. Na prática, μ_i e σ_i são desconhecidos e *estimados* pela média e desvio amostrais — e é justamente esse "estimados" que cria o risco de vazamento.

2.1 Vantagens

- Covariáveis tornam-se **adimensionais**: somem as unidades (kg, R$, anos).
- Resolve **discrepâncias numéricas** entre escalas — crucial para métodos baseados em distância (KNN, Aula 04) e em penalização (Lasso/Ridge, Aula 02), que de outro modo seriam dominados pelas variáveis de maior magnitude.
- Torna o **intercepto desnecessário** em regressão linear (dados centrados em zero).

2.2 Desvantagens

- Leve perda de **interpretabilidade**: os coeficientes passam a estar em "desvios-padrão", não nas unidades originais.
- μ_i e σ_i são **aprendidos dos dados** — exigindo cuidado para não vazar informação do teste (próxima seção).

Atenção

Nem todo método precisa de padronização, e são **três** categorias, não duas.
Árvores e florestas (Aula 05) comparam valores *dentro* de cada variável e são completamente indiferentes à escala — padronizar não muda nada além de gastar tempo. Já KNN e qualquer coisa penalizada precisam.
A terceira é menos óbvia: **LDA, QDA e Bayes ingênuo** (Aula 07) também são invariantes, e justamente *porque* estimam a covariância dos dados. Multiplicar a coluna j por c multiplica a linha e a coluna j de $\widehat{\Sigma}$ por c, e a forma quadrática $(\boldsymbol{x} - \widehat{\boldsymbol{\mu}})^\top \widehat{\Sigma}^{-1} (\boldsymbol{x} - \widehat{\boldsymbol{\mu}})$ sai idêntica — a padronização já está embutida na estimativa. Isso vale para os estimadores de máxima verossimilhança. No `scikit-learn`, o `GaussianNB` soma às variâncias um termo proporcional à maior delas, e o `reg_param` do QDA puxa a covariância na direção da identidade: os dois dependem da escala e quebram a invariância exata.
Saber *por que* cada método precisa (ou não) é melhor do que padronizar por reflexo — ainda que, na dúvida, deixar o `StandardScaler` no *pipeline* seja barato: nos invariantes ele não custa nada.

3 Vazamento de dados (*data leakage*)

Atenção: A armadilha

Se você aplicar o `StandardScaler` (ou qualquer transformação que aprende parâmetros) no *conjunto inteiro antes* de separar treino e teste, a média/desvio usados no treino **já contêm informação do teste**. As métricas deixam de medir o desempenho em dados *verdadeiramente novos* e ficam otimistas. Isso é **vazamento de dados**.

A regra correta:

1. separe treino e teste *primeiro*;
2. `fit` do scaler **só no treino** (aprende $\widehat{\mu}$, $\widehat{\sigma}$ do treino);
3. `transform` aplicado ao treino *e* ao teste com *esses mesmos* parâmetros.

O mesmo vale para *cada dobra* da validação cruzada (Aula 03): o scaler deve ser reajustado usando apenas o treino *daquela* dobra.

3.1 Quanto custa vazar? Duas medições

Essa recomendação costuma ser repetida sem número nenhum. Vale medir, e o resultado tem uma surpresa (Figura 1).

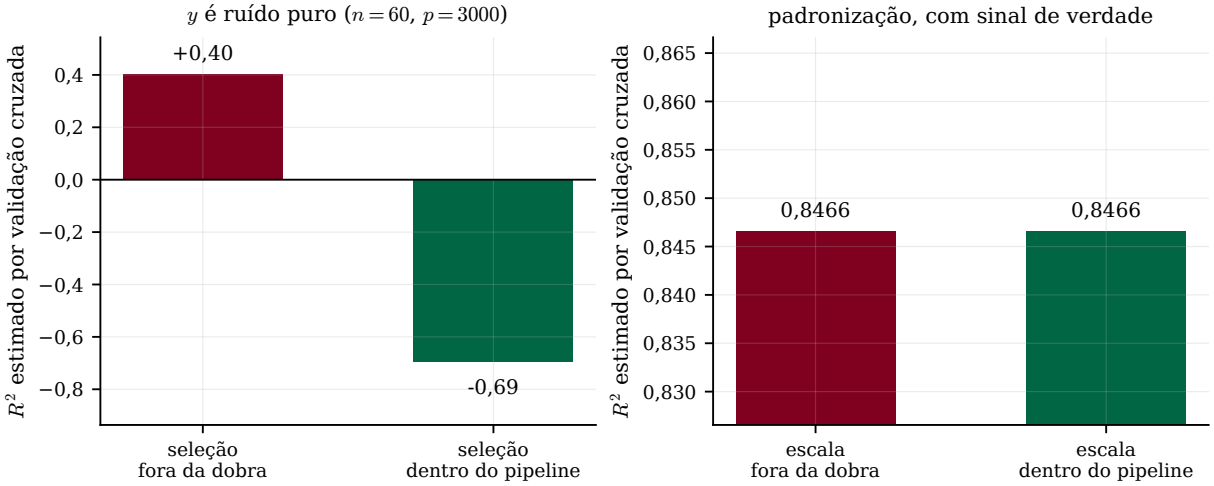


Figura 1: Dois vazamentos, medidos em 40 repetições, com validação cruzada de 5 dobras. **À esquerda**, seleção de variáveis: $n = 60$ observações, $p = 3000$ covariáveis e uma resposta y que é *ruído puro*, sem nenhuma relação com X — o R^2 verdadeiro é zero. Escolher as 20 covariáveis mais correlacionadas com y *antes* da validação cruzada produz $R^2 = +0{,}40$ *a partir de nada*. Fazendo a seleção dentro do *pipeline*, a CV devolve $-0{,}69$ e denuncia corretamente que não há sinal. **À direita**, padronização, num problema com sinal de verdade e escalas muito díspares: 0,8466 contra 0,8466 — iguais até a quarta casa decimal.

Ideia central: nem todo vazamento é igual

A lição do painel esquerdo é a que assusta: a seleção olhou 3000 colunas de ruído e ficou com as 20 que, *por acaso*, se pareciam com y naquela amostra. As dobras da CV vieram depois e encontraram essas 20 já escolhidas — inclusive nas observações que deveriam ser "novas". O modelo não aprendeu nada sobre o mundo; aprendeu sobre o próprio conjunto de teste. E note que $R^2 = +0{,}40$ é um número que ninguém questionaria num relatório. O painel direito é a lição complementar: a padronização vaza apenas duas estatísticas robustas por coluna, calculadas sobre dezenas de observações, e a diferença entre usar as do conjunto todo ou as do treino é numericamente desprezível. O aviso clássico não está errado — só não é ali que mora o perigo.

Atenção: a hierarquia que vale memorizar

Ordenados por gravidade:

1. **Grave:** escolher variáveis, hiperparâmetros ou o próprio modelo olhando dados que depois serão usados para avaliar. Inflação sem limite, como no painel esquerdo.
2. **Grave:** usar informação do futuro em dados temporais, ou ter a mesma unidade (paciente, cliente) em treino e teste.

3. **Leve:** padronizar, imputar pela média, codificar categóricas com o conjunto todo. Quase sempre inofensivo com n razoável — mas corrija assim mesmo, porque **custa zero**: é só pôr no *pipeline*.

4 *Pipelines*

Um `Pipeline` encadeia etapas (nome, transformador) terminadas por um estimador. Ao chamar:

- `pipe.fit(X_tr, y_tr)`: cada transformador faz `fit_transform` *no treino*, em sequência, e o estimador final é ajustado;
- `pipe.predict(X_te)`: cada transformador apenas `transform` (com os parâmetros aprendidos no treino) e o estimador prediz.

Assim, é *impossível* vazar o teste por construção: o teste nunca participa de nenhum `fit`.

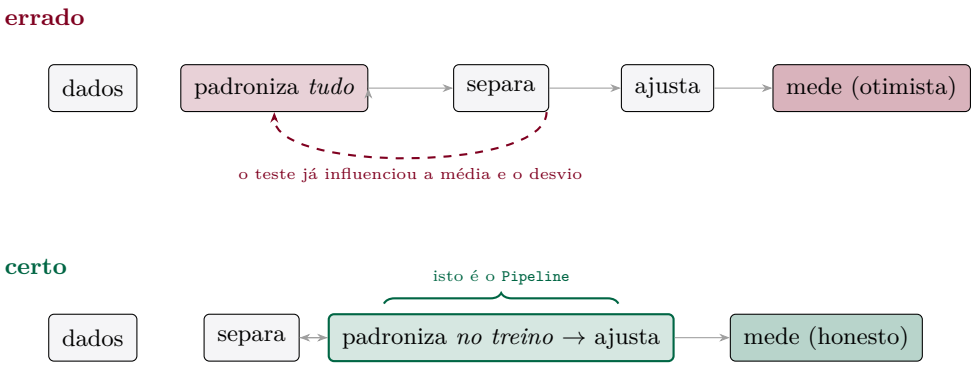


Figura 2: A mesma análise nas duas ordens. Em cima, a padronização acontece antes da separação, e os parâmetros $\widehat{\mu}, \widehat{\sigma}$ carregam informação das observações que só deveriam aparecer na hora de medir. Embaixo, tudo o que aprende fica *dentro* do bloco que só vê o treino — e esse bloco é exatamente o que o `Pipeline` representa em código.

```python
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Lasso
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, GridSearchCV

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=0)

pipe = Pipeline(steps=[("scaler", StandardScaler()),
                       ("lasso", Lasso())])
pipe.fit(X_tr, y_tr) # scaler aprende mu/sigma SO do treino
pipe.predict(X_te) # teste e apenas transformado
```

4.1 *Pipeline* + validação cruzada + busca de hiperparâmetros

O grande payoff: passar o *pipeline* inteiro ao `GridSearchCV`. Em cada dobra, todo o pré-processamento é reajustado corretamente, e a busca pelos hiperparâmetros (inclusive do scaler, se houver) fica livre de vazamento. Note a sintaxe `nomeEtapa__parametro` para acessar parâmetros de cada etapa.

```python
from sklearn.linear_model import ElasticNet

pipe_en = Pipeline([("scaler", StandardScaler()),
                    ("enet", ElasticNet())])
```

```
param_grid = {"enet__alpha": [0.001, 0.01, 0.1, 1, 5, 10, 50, 100],
              "enet__l1_ratio": [0.1, 0.5, 0.7, 0.9, 0.95, 0.99, 1]}

busca = GridSearchCV(pipe_en, param_grid, cv=5,
                     scoring="neg_mean_squared_error")
busca.fit(X_tr, y_tr) # CV SEM vazamento, etapa por etapa
print("melhores parametros:", busca.best_params_)
print("risco no teste:", -busca.score(X_te, y_te))
```

Atenção

Repare que a última linha usa `X_te`, que o `GridSearchCV` nunca viu. Isso é obrigatório: o `busca.best_score_` é o melhor resultado *de* 56 *combinações testadas*, e o máximo de 56 estimativas ruidosas é otimista por construção — mesmo com a CV feita corretamente. É a validação aninhada da Aula 03, na prática: a CV escolhe, o teste mede.

Ideia central

Encare o *pipeline* como "o modelo". Tudo o que você faria errado à mão — padronizar cedo demais, esquecer de aplicar a mesma transformação no teste, vazar na CV — o *pipeline* previne por construção. É a tradução em código das regras das Aulas 02–04.

5 Outros pré-processamentos comuns

Imputação preencher valores faltantes (`SimpleImputer` com média, mediana ou moda). A estatística de imputação também se aprende *só no treino*.

Codificação de categóricas
`OneHotEncoder` (cria *dummies*) ou `OrdinalEncoder`. Lembre (Aula 05) que árvores lidam com categóricas sem isso. Use `handle_unknown="ignore"`: uma categoria pode existir no teste e não no treino, e sem isso o `predict` quebra.

Escalas alternativas
`MinMaxScaler` (intervalo $[0, 1]$), `RobustScaler` (baseado em quantis, resistente a *outliers*).

Redução de dimensão
PCA como etapa do *pipeline* (Aula extra E2), também ajustado apenas no treino. Ele aprende direções de todas as colunas ao mesmo tempo, o que parece grave — mas **não olha o** Y, e por isso não consegue fabricar sinal a partir de ruído: a medição da Aula E2 o coloca na categoria *leve* desta hierarquia, junto com a padronização.

Quando diferentes colunas precisam de tratamentos diferentes (numéricas padronizadas, categóricas codificadas), usa-se o `ColumnTransformer`:

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer

prep = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                      ("sc", StandardScaler())]), colunas_numericas),
```

```
    ("cat", OneHotEncoder(handle_unknown="ignore"), colunas_categoricas),
])
modelo = Pipeline([("prep", prep), ("clf", Lasso())])
```

Note que o `ColumnTransformer` é, ele próprio, uma etapa de um *pipeline* maior, e que a coluna numérica passa por um *pipeline* aninhado. A estrutura é recursiva de propósito: qualquer análise, por mais complicada, cabe num único objeto com um `fit` e um `predict`.

O notebook `Aula prática 06` segue o fio principal desta aula: mede quais métodos são sensíveis à escala, põe um número no custo de padronizar fora da dobra (chega a 0,8342 contra 0,8341, com outra semente, e a mesma conclusão do painel direito da Figura 1), abre o `Pipeline` para conferir que o scaler aprendeu a média *do treino*, e fecha entregando o *pipeline* inteiro ao `GridSearchCV`. A `Lista prática 06` exercita o mesmo caminho, com outras sementes.

O vazamento por seleção de variáveis — o painel esquerdo da Figura 1 — e o vazamento por observações agrupadas não têm laboratório próprio: ficam por conta destas notas, e do Exercício 1 da `Lista teorica 06`, onde aparecem entre os seis casos a classificar. São os dois **graves** da hierarquia, e vale reler esta seção com cuidado justamente porque você não vai medi-los com as mãos.

Resumo

- Pré-processamento que aprende parâmetros é **parte do modelo**: ajuste só no treino, aplique ao teste (Fig. 2).
- **Padronização** (1): adimensionaliza, equaliza escalas (essencial para KNN e penalização, irrelevante para árvores), dispensa intercepto; custo: leve perda de interpretabilidade.
- **Vazamento** não é tudo igual (Fig. 1): seleção de variáveis ou de hiperparâmetros fora da dobra fabrica R^2 do nada; padronizar fora da dobra é, na prática, quase inofensivo. Corrija os dois — o *pipeline* torna isso gratuito.
- ***Pipeline***: encadeia tudo num único objeto e previne vazamento por construção; combine com `GridSearchCV` e ainda assim meça no teste.
- `ColumnTransformer` aplica tratamentos distintos a colunas distintas, e aninha *pipelines* dentro de *pipelines*.

Leitura recomendada. [ISLP] laboratórios dos Capítulos 5 e 6, onde os autores montam exatamente esses fluxos em Python; [AME] §2.1.2 (regressão linear na prática). Documentação do `scikit-learn`: *Pipelines and composite estimators* e *Common pitfalls and recommended practices*, esta última dedicada ao vazamento. Para o experimento da Figura 1, o texto de referência é Hastie, Tibshirani & Friedman, *The Elements of Statistical Learning*, §7.10.2 (“The Wrong and Right Way to Do Cross-validation”).

Para praticar. `Lista teorica 06.pdf` (teórica, com gabarito) e `Lista prática 06.ipynb` (prática, para completar as lacunas), nesta mesma pasta.